AMERICAN UNIVERSITY OF BEIRUT
MAROUN SEMAAN FACULTY OF ENGINEERING & ARCHITECTURE

EECE 503Q – DevSecOps
Project ShopCloud

E-Commerce Platform

1. Context & objective

You are working for a startup building ShopCloud, a lightweight e-commerce platform targeting customers in Europe and the United States. Your mission is to develop the application, design, provision, and automate the cloud infrastructure that makes it run reliably, securely, and cost-effectively at scale.

Think like a platform engineering team. Every decision you make — which compute model to use, how to segment your network, where to draw environment boundaries — must be justified on technical and economic grounds.

2. Platform overview

The table below describes each service from an operational standpoint. Read each description carefully. Your first task is to map each requirement to a concrete cloud service and justify your choice.

Service	Description
Product catalog	Public-facing storefront serving product listings, images, categories, and search results to anonymous and authenticated customers.
Shopping cart	Session-aware cart service that persists items across page loads.
Checkout & payment	Orchestrates the checkout flow: validates cart contents, confirms stock, and records the order. No real payment gateway integration — the service simulates payment confirmation.
Authentication	Handles customer registration, login, and session management. A separate admin authentication flow controls access to the inventory management panel.
Inventory admin panel	Internal web interface for warehouse staff and administrators to manage product listings, update stock levels, and process returns. Must never be reachable from the public internet — accessible only via a controlled internal network entry point.
Orders & inventory database	Relational database storing all orders, customer records, product inventory, and transaction history. Requires high availability with automatic failover, geographic redundancy, and regular automated snapshots.

Service	Description
Invoice generation	When an order is confirmed, a PDF invoice is automatically generated and emailed to the customer. This job must not block or delay the checkout response.
Dev & production environments	The platform must run in two fully isolated environments: production (live customers) and development (internal testing and CI/CD). A misconfiguration or failure in dev must have zero impact on production. Each environment is independently deployable.

3. Environment strategy

The platform must run in two fully isolated environments. The table below defines their purpose and isolation requirements.

Environment	Purpose	Isolation requirement
Production	Live customer traffic. No tolerance for unplanned downtime or data exposure.	Completely isolated at the network level. A failure or misconfiguration in dev must have zero impact on production.
Development	Internal testing, CI/CD validation, and integration checks. May use lower-cost, reduced-redundancy resources.	Separate network namespace. No shared credentials, secrets, or database connections with production.

4. Deliverables and Project phases

All deliverables are submitted as a single Git repository plus a live demo session. The repository must be runnable by the grading team from a clean AWS account.

Phases	Deliverable	Expected outcome
1	**Architecture and design (no implementation or code)**	• A **report** enclosing the fully labelled diagram showing all services, network boundaries, availability zones, and data flows. Each component must be justified: explain why you chose it over alternatives and what the cost implication is. • You should map each service to its AWS infrastructure (for example the catalog is running on the <X> AWS infrastructure). • A cost estimation (you may use the AWS calculator) to estimate the minimum investment to go live
2	**App development**	The full app codebase implemented on the designed infrastructure

Phases	Deliverable	Expected outcome
	Infrastructure as Code	Complete Terraform codebase that provisions the entire environment from scratch with a single command
	CI/CD pipeline	Automated pipeline (GitHub Actions or equivalent) that builds, tests, and deploys to the target environment on every push. Dev and production pipelines must be clearly separated. Include rollback instructions.
3	**Monitoring and Observability**	The needed monitoring setup and configuration to ensure high availabilty
	Security hardening	The security recommendations

- Phase 1 deliverables is only a report enclosing the infrastructure architecture, the mapping between the app services and the infrastructure, cost estimation and the logic supporting your design methodology
- While developing your code in phase 2, you should rely heavily on GitHub in your development collaboration. We will go over you GitHub project history to make sure efficient use of the GitHub platform

5. Grading and deadlines

- Phase 1: Tuesday April 7 midnight – 25% of the grade
- Phase 2: Tuesday April 22 midnight – 65% of the grade
- Phase 3: Tuesday April 28 midnight – 10% of the grade

6. Important note on AI use

App Code:

You may use any programming language and database technology of your choice. Since this is a DevOps-focused project, you are allowed to use AI **as a supporting tool** during application development. However, the code must not be fully generated by AI. You are responsible for your code and its quality, and you will be challenged to demonstrate your understanding of it.

AI Usage in DevOps:

The use of AI for DevOps-related tasks—including infrastructure design, Terraform, CI/CD pipelines, monitoring, and security—**is strictly prohibited**. You will be evaluated on these aspects, and any detected use of AI will result in a grade of zero for the entire project.